



SLTC

Learn. Research. Transform.

CCS3307 Data Warehousing Assignment 2

Project Report

Team Members:

22UG2-0013 Samadhi Jayasundara

22UG2-0017 Deshani Wijewardhana

22UG2-0068 Nethulya Sooriarachchi

22UG2-0119 Sanduni Chethana

22UG2-0271 Muthuni Nimshi

1.0. Introduction

Our chosen domain for this project is Retail. The reason to choose this domain was due to the fact that the modern retail industry is a hyper-competitive landscape characterized by slim profit margins, complex supply chains, and evolving customer expectations. In this environment, leveraging data is no longer a competitive advantage but a fundamental necessity for survival and growth.

Our dataset was taken from the Kaggle platform.

Retailers like the Superstore in our dataset detail information about transactional data daily, including sales records, customer information, product details, and shipping logistics. However, this raw data, often stored in systems designed for rapid transaction processing, is not structured for meaningful strategic analysis.

The primary challenge for "Superstore" is to transform its wealth of operational data into actionable intelligence. Key business questions that are currently difficult to answer include:

- Who are our most profitable customer segments?
- Which product categories are underperforming in specific regions?
- How do seasonal trends and promotional activities impact sales and profitability?
- What are the key drivers of customer churn or loyalty?

This report proposes the design, development, and implementation of a data warehouse solution made for Superstore. This solution will create a centralized, reliable, and analysis-ready repository of historical data.

By doing so, it will empower business analysts and decision-makers to move beyond simple operational reporting and engage in sophisticated performance analysis, trend identification, and predictive modeling. The ultimate goal is to provide the insights needed to optimize sales strategies, improve profitability, and enhance customer satisfaction.

2.0. Why Does SuperStore Need a Data Warehouse?

As per our assumption, the existing infrastructure at Superstore is likely built on an Online Transaction Processing (OLTP) system, similar to other retail stores. This system is optimized for managing daily work such as processing orders, updating inventory, and managing customer records with high speed and accuracy. However, its structure is fundamentally unsuited for the complex, large-scale queries required for business intelligence.

The need for a data warehouse comes from the limitations of using an OLTP system for analytical work:

- **Performance Degradation:** Running complex analytical queries, for example calculating the total profit for each product sub-category in the West region for the last three years, on a live OLTP database, would slow down critical business operations.
- **Data Structure:** The Current OLTP database is highly normalized to avoid data redundancy and ensure transactional integrity. While efficient for operations, this structure makes querying for analysis extremely complex, requiring numerous table joins that are slow and resource-intensive.
- **Historical Data:** OLTP systems typically only store current data. A data warehouse, on the other hand, is designed to store and manage large volumes of historical data, which is essential for trend analysis and forecasting.
- **Data Integration:** Data may come from multiple sources, and a data warehouse serves as the central point for integrating this disparate data into a single, consistent format.

To address these challenges, we propose an Online Analytical Processing (OLAP) system in the form of a data warehouse. This system would support decision-making and business intelligence.

3.0. Proposed Architecture

For our Superstore dataset, we propose a Three-Tier Data Warehouse Architecture. This is a standard and highly effective industry model that separates the major functions of the data warehouse into distinct logical layers. This separation makes the system more manageable, scalable, and easier to maintain. The architecture consists of a bottom tier for data storage and processing, a middle tier for analytical processing, and a top tier for user-facing data presentation.

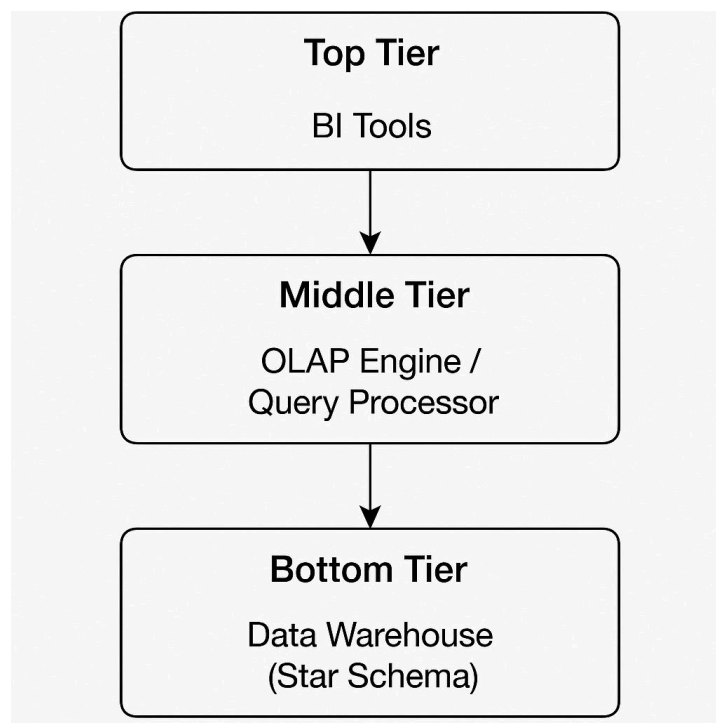
The Three Tiers:

1. **Bottom Tier (Data Warehouse Server):** This foundational layer is the backend of the architecture. It is responsible for all data extraction, transformation, and loading (ETL) processes. For our project, this tier consists of the source `Superstore.csv` file and the SQL Server database that acts as both the staging area and the final data warehouse repository. The ETL pipeline is executed here to cleanse the raw data and load it into our structured schema.
2. **Middle Tier (OLAP Server):** This tier is the analytical engine of the data warehouse. It presents the data from the bottom tier in a structured, multidimensional format, optimized for analysis. We implement this using a **Relational OLAP (ROLAP)** model, where our dimensional **Star Schema** (with its central fact table and surrounding dimension tables) resides directly within the SQL Server database. This schema allows for OLAP operations like slicing, dicing, and drilling down, enabling business analysts to explore the data from various perspectives without needing a separate, proprietary OLAP server.
3. **Top Tier (Front-End Client):** This is the user-facing presentation layer. It provides the interface for business users to access, analyze, and visualize the data from the OLAP server. For our solution, this tier is composed of Business Intelligence (BI) tools such as Power BI and Tableau.

These tools connect to our middle-tier schema to generate interactive dashboards, reports, and perform ad-hoc queries, turning complex data into actionable business insights.

This three-tier structure provides several key benefits for the Superstore project:

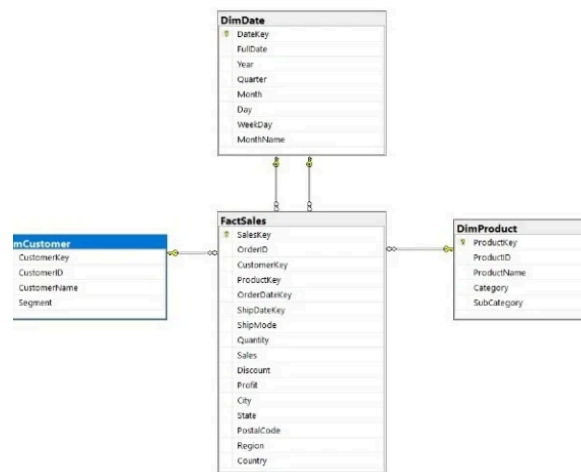
- **Modularity and Scalability:** Each tier can be developed, managed, and scaled independently. For example, we can enhance the ETL process in the bottom tier without affecting the reports built in the top tier.
- **Clear Separation of Concerns:** The architecture clearly separates the complex task of data processing (bottom tier) from data analysis (middle tier) and data visualization (top tier), making the entire system easier to understand and maintain.
- **Performance:** By structuring the data into an optimized Star Schema in the middle tier, we ensure that queries from the front-end BI tools are executed quickly and efficiently, providing a responsive experience for business users.
- **Flexibility:** The top tier is decoupled from the underlying database, allowing the organization to use a variety of reporting tools (like Power BI, Tableau, or others) to connect to the middle-tier OLAP server, depending on their needs and preferences.



4.0. Data Modelling and Schema Design

For this data warehouse, we selected a **Dimensional Modeling** approach over traditional Entity-Relationship (E-R) modeling. While E-R modeling is ideal for operational (OLTP) databases where the goal is to eliminate redundancy and ensure transactional integrity, it results in a highly normalized structure that is inefficient for analytics. Dimensional modeling, in contrast, is designed specifically for data warehousing (OLAP) to optimize for fast query performance and ease of understanding for business users.

The chosen dimensional model for this project is the Star Schema. This schema is characterized by a central fact table that is directly linked to several de-normalized dimension tables.



At the center of our design is the FactSales table, which captures the quantitative measures from the business process, such as sales amount, quantity, discount, and profit. Surrounding it are four dimension tables that provide the descriptive context for analysis:

- A Customer dimension (DimCustomer) storing customer-specific attributes like IDs, names, and segments.
- A Product dimension (DimProduct) describing the product hierarchy, including categories, sub-categories, and product names.
- A Location dimension (DimLocation) containing all geographical information such as the country, region, state, and city.
- A Date dimension (DimDate) which is a pre-populated table that breaks down timelines into year, quarter, month, and day for both orders and shipments.

This star schema structure is highly effective because it simplifies queries by reducing the number of complex joins required to link data. This results in significantly faster performance for the analytical tools like Power BI and ensures that business users can intuitively explore the data.

4.1. Fact Table

The fact table contains the numerical measures of a business process. Our fact table will capture the key performance indicators (KPIs) for each sales transaction.

Table:

FactSales

Column Name	Data Type	Description	Type
SalesKey	INT (PK)	Surrogate Primary Key	-
OrderDateKey	INT (FK)	Foreign key to DimDate	Foreign Key
CustomerKey	INT (FK)	Foreign key to DimCustomer	Foreign Key
ProductKey	INT (FK)	Foreign key to DimProduct	Foreign Key
LocationKey	INT (FK)	Foreign key to DimLocation	Foreign Key
Sales	DECIMAL(18, 4)	The total sale amount	Degenerate Dimension
Quantity	INT	The number of units sold	Measure
Discount	DECIMAL(18, 4)	The discount applied	Measure
Profit	DECIMAL(18, 4)	The profit from the sale	Measure
OrderID	NVARCHAR(255)	Original order ID from source	Degenerate Dimension

4.2. Dimension Tables

Dimension tables store the descriptive attributes (the "who, what, where, when") of the sales data. They provide the context for the measures in the fact table. We use surrogate keys (integer primary keys managed by the data warehouse) to decouple the warehouse from the operational system's keys and to handle changes in descriptive data over time.

Table: DimDate

Column Name	Data Type	Description
DateKey	INT (PK)	Surrogate Primary Key
Date	DATE	The full date
Year	INT	The calendar year

Quarter	INT	The quarter (1-4)
Month	INT	The month (1-12)
MonthName	NVARCHAR(20)	The name of the month
Day	INT	The day of the month
DayOfWeek	NVARCHAR(20)	The name of the day of the week

The other tables include DimCustomer, DimProduct, and DimLocation.

5.0. ETL Process

The ETL process is the backbone of the data warehouse, responsible for populating it with clean, reliable data. We will use SQL Server Integration Services (SSIS) or a series of SQL scripts to perform these steps.

1. Extract

- **Source:** The Superstore.csv file.
- **Process:** An SSIS "Data Flow Task" will be configured to read the entire dataset from the flat file source. All columns will be extracted initially into the staging area as strings to prevent data type errors during the initial pull.

2. Transform This is the most complex stage, performed in the staging area.

- **Data Cleansing:**
 - **Handling Nulls:** The Postal Code column has missing values. These will be replaced with a default value (e.g., '00000') or derived based on the city and state if possible.
 - **Standardization:** The Order Date and Ship Date columns will be converted from their string format (e.g., "dd/mm/yyyy") to a standard DATE format. Text fields like Category and Segment will be standardized (e.g., using UPPER() and TRIM() functions) to ensure consistency.
- **Business Rule Application:**
 - **Profitability Flag:** A new column, IsProfitable, could be derived (CASE WHEN Profit > 0 THEN 'Yes' ELSE 'No' END) for simplified analysis in dashboards.
- **Data Integration & Restructuring:**

- **De-duplication:** We will create distinct lists of Customers, Products, and Locations from the source data to populate our dimension tables. This involves selecting DISTINCT values for the relevant columns (CustomerID, CustomerName, Segment, etc.).
- **Surrogate Key Generation:** For each dimension (DimCustomer, DimProduct, etc.), we will generate new integer surrogate keys using the IDENTITY property in SQL Server. Lookup transformations will be used to find the correct surrogate key for each dimension based on the natural key (e.g., CustomerID) from the source data and add it to the data flow before loading the fact table.
- **Date Dimension Creation:** A script will be used to pre-populate the DimDate table with all dates for a given range of time, ensuring every possible date has an entry.

3. Load

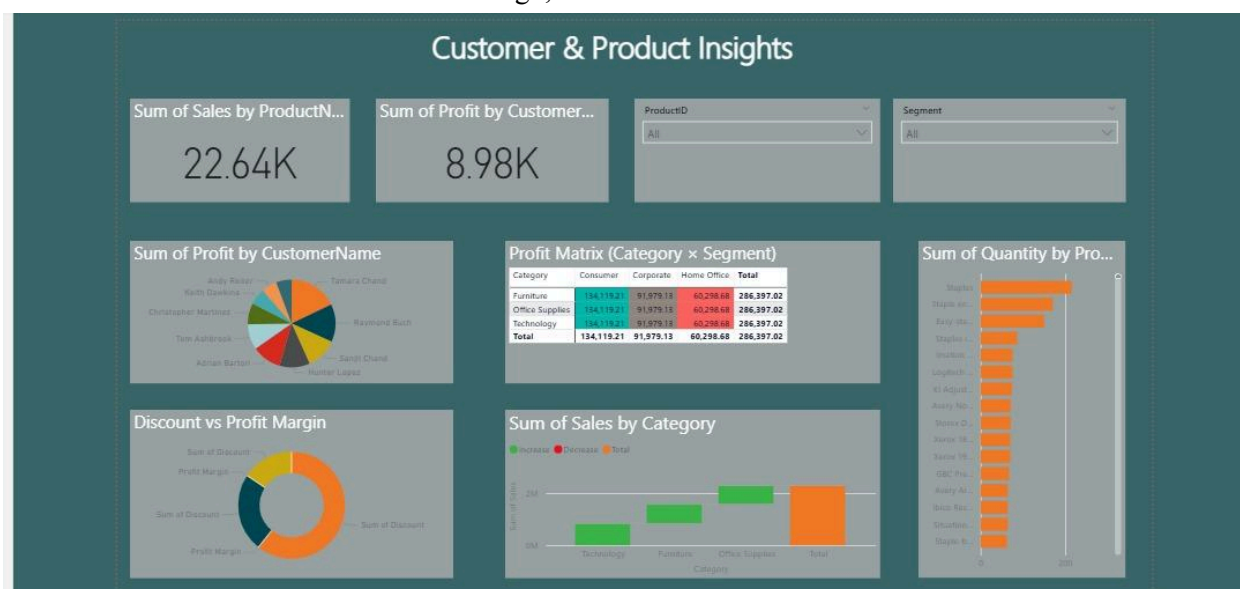
- **Process:** The transformed data is loaded into the final destination tables in the data warehouse.
- **Loading Order:** The dimension tables (DimCustomer, DimProduct, DimLocation, DimDate) must be loaded *first*. This ensures that when the FactSales table is loaded, all the required foreign keys are already present in the dimension tables, maintaining referential integrity.
- **Loading Strategy:** An initial full load will populate the warehouse. For ongoing updates, an incremental loading strategy would be implemented, where only new or changed records from the source system are processed and loaded, typically on a nightly basis.

6.0. Dashboards in Power BI



The Sales and Profit Overview dashboard serves as a strategic analytical interface, designed in Power BI to empower dynamic decision-making rather than static reporting. The top-line KPIs offer an immediate, quantitative snapshot of the business's core performance drivers, including Total Sales (\$2.30M) and overall Profitability (\$286.40K). The dashboard shows the immediate product portfolio analysis via the "Sales by Category" chart, while the "Profit by Segment" visual allows for sophisticated customer profitability assessment which is a critical input for guiding marketing strategy.

We also worked on a second dashboard design,



This Customer and Product Insights dashboard is designed for a more granular, diagnostic analysis of profitability drivers. It features a detailed profit matrix that cross-analyzes product categories against customer segments to pinpoint the most lucrative business areas. The dashboard also includes specific visuals to assess the impact of discounts on profit margins, identify the most profitable individual customers, and rank products by the quantity sold. Using the interactive filters, managers can drill down into specific products or segments to investigate the root causes behind business performance.

7.0. Data Quality Management

Maintaining high data quality is paramount for user trust and the success of the data warehouse. Our data quality strategy involves proactive and reactive measures integrated into the ETL process.

- **Data Profiling:** Before any ETL development, the source dataset was profiled to identify potential quality issues like missing values (e.g., Postal Code), inconsistent formatting, and outliers in numerical fields (Sales, Profit).
- **ETL-Based Validation Rules:** The following rules will be embedded within the SSIS transformation logic:

- **Type Validation:** Ensure that numerical columns like Sales and Quantity do not contain non-numeric characters.
- **Range Checks:** Flag any sales records where Discount is greater than a plausible limit (e.g., 80%) for manual review.
- **Referential Integrity:** The ETL process will ensure that every record loaded into FactSales has a valid, existing key in each of the corresponding dimension tables.
- **Error Handling and Logging:** Any row from the source data that fails a validation rule will be redirected to an "Error Table" in the staging database instead of being discarded. This table will log the record and the specific reason for the failure. A data steward can then review these errors, correct the source data if necessary, and re-process the failed records.

8.0. Deployment and Maintenance Plan

A structured plan is required to deploy the data warehouse into a production environment and ensure its long-term reliability.

Deployment Plan:

1. **Environment Setup:** We will use a three-environment approach:
 - **Development:** Where developers build and test the database and ETL packages.
 - **Testing/QA:** A separate environment where business users can test the dashboards and validate the data against the source systems before deployment.
 - **Production:** The live environment used by the business for analysis.
2. **Migration:** The deployment process will involve migrating the SQL Server database schema and the finalized SSIS packages from the Test environment to the Production environment. The initial data load will be performed during a low-traffic period.

Maintenance Plan:

- **Daily:**
 - Monitor the execution of nightly ETL jobs to ensure they complete successfully.
 - Review error logs for any failed records and notify data stewards.
- **Weekly:**
 - Review database performance metrics and query execution times. Identify and optimize any slow-running queries.
 - Check disk space usage and plan for future capacity needs.

- **Monthly:**
 - Perform index maintenance (rebuild/reorganize) to ensure optimal query performance.
 - Archive or purge old log files.

9.0. Conclusion

Ultimately, this project proves just how powerful a well-designed data warehouse can be for a business like Superstore. By building a reliable system with a clear Star Schema, we've created a single, trustworthy source for all their sales data, ensuring everyone is on the same page. But the real win here is not just the technology; it's the insights we've uncovered that were impossible to see before. For the first time, the business can clearly see that the Central region is bleeding profits on furniture, that corporate clients are their most valuable customers, and that they're actually losing money on products like tables and bookcases.

In the end, what we have built is more than just a technical database; it is a strategic tool that gives Superstore the clarity to make smarter, faster decisions, react to market trends, and find real opportunities for growth.